Django Database & ORM — Complete Notes A to Z

Django 6.0 | Beginner to Advanced | Simple English + Bangla Explanation

Table of Contents

1. Database Basics
2. Django ORM
3. Project Setup
4. Django Models and Fields
5. Migrations
6. CRUD Operations
7. QuerySets and Filtering
8. Field Lookups and Q Objects
9. Model Relationships
10. Ordering and Limiting Results
11. select_related() and prefetch_related()
12. Aggregation and Annotation
13. F Expressions
14. Conditional Expressions
15. Subquery and Exists
16. Managers and Custom QuerySets
17. Database Transactions
18. Raw SQL
19. Query Optimization and Indexes
20. Bulk Operations
21. Multiple Databases
22. Fixtures
23. Signals
24. Validation, Constraints and Security
25. Interview Questions
26. Quick ORM Cheat Sheet
27. Learning Roadmap

Chapter 1: Database Basics

1.1 What is a Database?

A database is a system for storing, finding, updating, and deleting information.

Examples:

- Student names and emails
- Product information
- Customer orders
- User accounts
- Examination results

Popular relational databases include PostgreSQL, MySQL, SQLite, and Oracle.

বাংলা: Database হলো এমন একটি ব্যবস্থা যেখানে তথ্য সংরক্ষণ, খোঁজা, পরিবর্তন এবং মুছে ফেলা যায়।

1.2 What is a Relational Database?

A relational database stores information in tables. A table contains rows and columns.

Term	Meaning
Table	A collection of related data
Row	One record
Column	One field or attribute
Primary key	Uniquely identifies a record
Foreign key	Connects records across tables
Constraint	A rule that data must

	follow
Index	Can help the database find data faster

Example:

id	name	email
1	Rahim	rahim@example.com
2	Karim	karim@example.com

Here, `id` can be the primary key.

বাংলা: Primary key প্রতিটি record-কে আলাদাভাবে শনাক্ত করে। Foreign key একটি table-এর record-কে অন্য table-এর record-এর সঙ্গে যুক্ত করে।

1.3 What is SQL?

SQL means Structured Query Language. It is used to work with relational databases.

```
SELECT * FROM students_student;

SELECT * FROM students_student
WHERE age >= 18;

INSERT INTO students_student (name, age)
VALUES ('Rahim', 20);

UPDATE students_student
SET age = 21
WHERE id = 1;

DELETE FROM students_student
WHERE id = 1;
```

The exact table name depends on the model and database configuration.

বাংলা: ORM ব্যবহার করলেও SQL শেখা দরকার, কারণ query debugging ও optimization-এর সময় SQL বুঝতে সুবিধা হয়।

1.4 What is ACID?

- **Atomicity:** All operations succeed, or the transaction is rolled back.
- **Consistency:** Data follows database rules.
- **Isolation:** Concurrent transactions follow the database's isolation rules.
- **Durability:** Committed changes are preserved according to the database's guarantees.

Example: A bank transfer should not subtract money from one account without adding it to the other. A transaction helps keep related database changes consistent.

Chapter 2: Django ORM

2.1 What is ORM?

ORM means Object-Relational Mapping. Django ORM lets you work with a relational database using Python models and query methods.

SQL:

```
SELECT * FROM students_student WHERE age >= 18;
```

Django ORM:

```
Student.objects.filter(age__gte=18)
```

বাংলা: ORM Python object ও database table-এর মধ্যে সংযোগ তৈরি করে। সব query হাতে SQL লিখে করতে হয় না।

2.2 Benefits of Django ORM

- Write many database queries using Python.
- Manage tables through models and migrations.
- Work with related models.
- Build queries by chaining QuerySets.
- Use parameterized queries for many database operations.
- Switch database backends more easily, although features can differ by backend.

2.3 View the SQL Query

```
students = Student.objects.filter(age__gte=18)
print(students.query)
```

This displays the SQL representation of the QuerySet. It does not normally execute the query by itself.

বাংলা: ORM কী SQL তৈরি করছে তা দেখলে query বোঝা ও performance সমস্যা খুঁজে বের করা সহজ হয়।

Chapter 3: Project Setup

3.1 Create a Virtual Environment and Install Django

```
python -m venv .venv
```

Activate on Windows:

```
.venv\Scripts\activate
```

Activate on macOS/Linux:

```
source .venv/bin/activate
```

Install Django 6.0:

```
python -m pip install "Django>=6.0,<6.1"
```

Create the project and app:

```
django-admin startproject config .
python manage.py startapp students
python manage.py migrate
python manage.py runserver
```

Check the installed version:

```
python -m django --version
```

3.2 Register the App

In `config/settings.py`:

```
INSTALLED_APPS = [
    # Default Django apps
    "students",
]
```

Keep the default Django applications in `INSTALLED_APPS`; add `"students"` to the existing list rather than replacing the defaults.

Django needs the app registered to discover and manage its models and migrations as part of the project.

3.3 SQLite Configuration

A new Django project normally uses SQLite:

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

SQLite is useful for learning and many small projects.

3.4 PostgreSQL Configuration

Install a PostgreSQL driver compatible with your environment. Keep credentials in environment variables or a secret manager, not in source code.

```
import os

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.environ["DB_NAME"],
        "USER": os.environ["DB_USER"],
        "PASSWORD": os.environ["DB_PASSWORD"],
        "HOST": os.environ.get("DB_HOST", "localhost"),
        "PORT": os.environ.get("DB_PORT", "5432"),
    }
}
```

বাংলা: Local learning-এর জন্য SQLite সহজ। Production application-এ PostgreSQL বহুল ব্যবহৃত। Database password Git-এ commit করবে না।

Chapter 4: Django Models and Fields

4.1 What is a Model?

A model is a Python class that describes data stored in a database table.

In `students/models.py`:

```
from django.db import models


class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    age = models.PositiveIntegerField(default=18)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name
```

Explanation:

- `models.Model` : makes this a Django model.
- `CharField` : stores limited-length text.
- `EmailField` : provides email-oriented validation.
- `unique=True` : prevents duplicate email values at the database level.
- `default=18` : supplies a default value.
- `auto_now_add=True` : sets the timestamp when the object is first created.
- `__str__()` : returns a human-readable representation.

Django normally adds a primary key automatically if one is not defined.

বাংলা: Model হলো database table-এর Python representation। Field দিয়ে table-এর column ও তার type নির্ধারণ করা হয়।

4.2 Common Model Fields

Field	Use
`CharField(max_length=...)`	Limited-length text
`TextField()`	Longer text
`IntegerField()`	Integer

`PositiveIntegerField()`	Non-negative integer-oriented field
`DecimalField(max_digits=10, decimal_places=2)`	Precise decimal values, such as money
`FloatField()`	Floating-point values
`BooleanField()`	True/False
`DateField()`	Date
`DateTimeField()`	Date and time
`EmailField()`	Email
`URLField()`	URL
`UUIDField()`	UUID
`JSONField()`	JSON data
`FileField(upload_to="files/")`	File path/reference

`ImageField(upload_to="images/")`	Image path/reference and image validation support

Use `DecimalField` for money when decimal precision matters. `FloatField` uses floating-point representation and may introduce precision differences.

4.3 `null=True` vs `blank=True`

- `null=True` : the database can store SQL `NULL` .
- `blank=True` : the field may be empty during Django validation.

```
middle_name = models.CharField(
    max_length=100,
    blank=True,
)

age = models.IntegerField(
    null=True,
    blank=True,
)
```

For string fields, Django projects usually prefer an empty string (`""`) instead of using both `NULL` and an empty string to represent missing text.

বাংলা: `null` database storage-এর বিষয়; `blank` validation-এর বিষয়। দুটো এক নয়।

4.4 `default`, `unique`, and `db_index`

```
status = models.CharField(
    max_length=20,
    default="pending",
)

email = models.EmailField(unique=True)

slug = models.SlugField(db_index=True)
```

- `default` : value না দিলে কোন মান ব্যবহৃত হবে।

- `unique=True` : duplicate value আটকায়।
- `db_index=True` : field-এর ওপর database index তৈরি করে।

Indexes can improve some reads, but they use storage and may slow down writes.

4.5 Meta Options

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["name"]
        verbose_name = "Student"
        verbose_name_plural = "Students"
        indexes = [
            models.Index(fields=["created_at"]),
        ]
```

- `ordering` : default ordering of results.
- `verbose_name` : singular display name.
- `verbose_name_plural` : plural display name.
- `indexes` : database indexes declared for the model.

4.6 Model Methods

```
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )

    def __str__(self):
        return self.name

    def is_expensive(self):
        return self.price >= 1000
```

`is_expensive()` is a Python method, not a database field.

4.7 Choices

```
class Order(models.Model):
    class Status(models.TextChoices):
        PENDING = "pending", "Pending"
        PAID = "paid", "Paid"
        CANCELLED = "cancelled", "Cancelled"

    status = models.CharField(
        max_length=20,
        choices=Status.choices,
        default=Status.PENDING,
    )
```

The first value is stored in the database; the second is the display label. Choices help validation and display. Use database constraints when you need a database-level rule.

Chapter 5: Migrations

5.1 What is a Migration?

A migration is a version-controlled instruction for changing database structure. If you add a model field, a migration can add the corresponding database column.

5.2 Important Commands

```
python manage.py makemigrations
python manage.py migrate
python manage.py showmigrations
python manage.py sqlmigrate students 0001
```

Command	Purpose
`makemigrations`	Creates migration files from model changes

`migrate`	Applies migrations to the database
`showmigrations`	Shows migration status
`sqlmigrate`	Shows SQL for a migration

5.3 Migration Workflow

1. Change `models.py`.
2. Run `makemigrations`.
3. Review the generated migration.
4. Run `migrate`.
5. Run tests and verify the result.

Keep migration files in version control. Before production schema changes, consider backups, data migrations, deployment order, and rollback strategy.

বাংলা: `makemigrations` file তৈরি করে; `migrate` database-এ পরিবর্তন প্রয়োগ করে। দুটি command একই কাজ করে না।

Chapter 6: CRUD Operations

CRUD means Create, Read, Update, and Delete.

6.1 Create

```
student = Student.objects.create(
    name="Rahim",
    email="rahim@example.com",
    age=20,
)
```

Alternative:

```
student = Student(
    name="Karim",
    email="karim@example.com",
    age=21,
)

student.save()
```

`create()` creates and saves an object. Instantiating a model does not save it until `save()` is called.

6.2 Read

```
Student.objects.all()
Student.objects.get(pk=1)
Student.objects.filter(is_active=True)
Student.objects.exclude(age__lt=18)
```

- `all()` : all records in the QuerySet.
- `get()` : expects exactly one object.
- `filter()` : returns a QuerySet of matches.
- `exclude()` : returns a QuerySet excluding matches.

`get()` raises `DoesNotExist` if none match and `MultipleObjectsReturned` if multiple objects match.

```
try:
    student = Student.objects.get(pk=1)
except Student.DoesNotExist:
    student = None
```

6.3 Update One Object

```
student = Student.objects.get(pk=1)
student.age = 22
student.save(update_fields=["age"])
```

6.4 Update Multiple Objects

```
Student.objects.filter(
    is_active=False
).update(is_active=True)
```

`QuerySet.update()` changes matching rows directly. It does not call each instance's `save()` method or normal save signals.

6.5 Delete

```
student = Student.objects.get(pk=1)
student.delete()
```

Delete multiple objects:

```
Student.objects.filter(is_active=False).delete()
```

Always check the filter before deleting data. Django's deletion collector may also delete related objects according to relationship rules.

6.6 `get_or_create()`

```
student, created = Student.objects.get_or_create(
    email="rahim@example.com",
    defaults={
        "name": "Rahim",
        "age": 20,
    },
)
```

It returns a matching object or creates one. `created` indicates whether a new object was created. Database uniqueness constraints remain important under concurrent requests.

6.7 `update_or_create()`

```
student, created = Student.objects.update_or_create(
    email="rahim@example.com",
    defaults={
        "name": "Rahim Updated",
        "age": 21,
    },
)
```

It updates a matching object or creates one if none matches.

বাংলা: CRUD হলো database কাজের ভিত্তি। `get()` একটি object-এর জন্য, `filter()` এক বা একাধিক result-এর জন্য ব্যবহার করো।

Chapter 7: QuerySets and Filtering

7.1 What is a QuerySet?

A QuerySet represents a database query and its results.

```
students = Student.objects.filter(age__gte=18)
```

QuerySets are usually lazy: creating one does not normally execute the database query immediately.

7.2 QuerySet Chaining

```
students = (
    Student.objects
    .filter(is_active=True)
    .filter(age__gte=18)
    .order_by("name")
)
```

Multiple filter conditions are normally combined using `AND`.

7.3 Common Filters

```
Student.objects.filter(name="Rahim")
Student.objects.filter(age__gte=18)
Student.objects.filter(age__range=(18, 25))
Student.objects.filter(name__startswith="R")
Student.objects.filter(email__icontains="example.com")
Student.objects.filter(created_at__isnull=False)
```

7.4 QuerySet Evaluation

Operations that can evaluate a QuerySet include:

```
list(students)
len(students)
bool(students)

for student in students:
    print(student.name)
```

An evaluated QuerySet may cache its results. A newly constructed QuerySet does not automatically share that cache.

Use `exists()` when you only need to know whether matches exist, and `count()` when you need the number of matching rows.

বাংলা: QuerySet lazy হওয়ায় query প্রয়োজন হওয়ার আগে database access নাও হতে পারে। এতে query chain করে একসঙ্গে শর্ত তৈরি করা যায়।

Chapter 8: Field Lookups and Q Objects

8.1 Common Field Lookups

General pattern:

```
Model.objects.filter(field__lookup=value)
```

Lookup	Meaning	Example
`exact`	Exact match	`name__exact="Rahim"`
`iexact`	Case-insensitive exact match	`name__iexact="rahim"`
`contains`	Contains text	`name__contains="ah"`
`icontains`	Case-insensitive contains	`name__icontains="rah"`
`gt`	Greater than	`age__gt=18`
`gte`	Greater than or equal	`age__gte=18`
`lt`	Less than	`age__lt=18`

`lte`	Less than or equal	`age__lte=18`
`in`	Value is in a list	`id__in=[1, 2, 3]`
`range`	Value is in a range	`age__range=(18, 25)`
`isnull`	Checks SQL NULL	`created_at__isnull=True`
`startswith`	Starts with text	`name__startswith="R"`
`endswith`	Ends with text	`name__endswith="m"`

8.2 Q Objects

Q objects help build complex conditions.

```
from django.db.models import Q

# OR
students = Student.objects.filter(
    Q(age__lt=18) | Q(age__gt=60)
)

# AND
students = Student.objects.filter(
    Q(is_active=True) & Q(age__gte=18)
)

# NOT
students = Student.objects.filter(
    ~Q(name__icontains="test")
)
```

Symbols:

- `|` means OR.
- `&` means AND.
- `~` means NOT.

Search by name or email:

```
search = "rahim"

students = Student.objects.filter(
    Q(name__icontains=search)
    | Q(email__icontains=search)
)
```

বাংলা: সাধারণ filter arguments সাধারণত AND হিসেবে যুক্ত হয়। OR/NOT condition-এর জন্য `Q` object খুব কাজে লাগে।

Chapter 9: Model Relationships

9.1 ForeignKey — One-to-Many

One author can have many posts, while each post refers to one author.

```
class Author(models.Model):
    name = models.CharField(max_length=100)

    def __str__(self):
        return self.name


class Post(models.Model):
    author = models.ForeignKey(
        Author,
        on_delete=models.CASCADE,
        related_name="posts",
    )
    title = models.CharField(max_length=200)
    body = models.TextField()
```

Usage:

```
author = Author.objects.get(pk=1)
author.posts.all()
```

```
post = Post.objects.get(pk=1)
post.author
```

`on_delete` Options

- `CASCADE`: deletes related objects when the referenced object is deleted.
- `PROTECT`: prevents deletion when protected related objects exist.
- `RESTRICT`: restricts deletion under relationship conditions.
- `SET_NULL`: sets the foreign key to NULL; requires `null=True`.
- `SET_DEFAULT`: sets the foreign key to its default.
- `DO_NOTHING`: Django takes no automatic action; the database may raise an error.

Choose the behavior based on business requirements.

9.2 OneToOneField

```
class Profile(models.Model):
    student = models.OneToOneField(
        Student,
        on_delete=models.CASCADE,
        related_name="profile",
    )
    bio = models.TextField(blank=True)
```

Usage:

```
student = Student.objects.get(pk=1)
profile = student.profile
```

Each Student can have at most one Profile through this relationship. If no Profile exists, reverse access can raise a related-object exception.

9.3 ManyToManyField

A student can take many courses, and a course can have many students.

```
class Course(models.Model):
    title = models.CharField(max_length=100)


class Learner(models.Model):
    name = models.CharField(max_length=100)
    courses = models.ManyToManyField(
        Course,
        related_name="learners",
```

```
        blank=True,
    )
```

Usage:

```
learner = Learner.objects.get(pk=1)
course = Course.objects.get(pk=1)

learner.courses.add(course)
learner.courses.remove(course)
learner.courses.set([course])
learner.courses.clear()
```

Django normally creates an intermediary table for a Many-to-Many relationship.

9.4 Through Model

Use a through model when the relationship has its own data, such as enrollment date or grade.

```
class Enrollment(models.Model):
    learner = models.ForeignKey(
        Learner,
        on_delete=models.CASCADE,
    )
    course = models.ForeignKey(
        Course,
        on_delete=models.CASCADE,
    )
    enrolled_at = models.DateTimeField(auto_now_add=True)
    grade = models.CharField(max_length=5, blank=True)

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["learner", "course"],
                name="unique_learner_course",
            )
        ]
```

You can connect this model to a `ManyToManyField` using `through="Enrollment"`.

বাংলা: `ForeignKey` সাধারণত one-to-many, `OneToOneField` one-to-one এবং `ManyToManyField` many-to-many সম্পর্ক তৈরি করে।

Chapter 10: Ordering and Limiting Results

10.1 Ordering

```
Student.objects.order_by("name")
Student.objects.order_by("-created_at")
Student.objects.order_by("name", "-age")
```

A minus sign means descending order.

10.2 Limiting Results

```
Student.objects.all()[:10]
Student.objects.all()[10:20]
```

These slices normally translate to SQL `LIMIT` and `OFFSET`. Large offsets can become expensive; cursor or keyset pagination may be useful for some workloads.

10.3 `values()` and `values_list()`

```
Student.objects.values("id", "name", "email")
Student.objects.values_list("name", flat=True)
```

`values()` returns dictionaries containing selected fields. `values_list()` returns tuples by default, or a single-value iterable with `flat=True`.

10.4 `exists()`, `count()`, and `distinct()`

```
Student.objects.filter(is_active=True).exists()
Student.objects.count()
Post.objects.values("author_id").distinct()
```

- `exists()`: checks whether any matching records exist.
- `count()`: counts matching records.
- `distinct()`: removes duplicate result rows.

Some `distinct()` behavior depends on the database backend and query structure.

Chapter 11: `select_related()` and `prefetch_related()`

11.1 The N+1 Query Problem

```
posts = Post.objects.all()
```

```
  for post in posts:
      print(post.author.name)
```

Django may execute one query for Posts and additional queries for individual Authors. This is an example of the N+1 query problem.

11.2 `select_related()`

Use it mainly for ForeignKey and OneToOne relationships:

```
  posts = Post.objects.select_related("author")

  for post in posts:
      print(post.author.name)
```

It uses SQL JOINs to retrieve related single-valued objects efficiently.

11.3 `prefetch_related()`

Use it mainly for related collections:

```
  authors = Author.objects.prefetch_related("posts")

  for author in authors:
      for post in author.posts.all():
          print(post.title)
```

It normally uses additional queries and joins the results in Python.

Method	**Typical use**	**Approach**
`select_related()`	ForeignKey, OneToOne	SQL JOIN
`prefetch_related()`	Reverse ForeignKey, ManyToMany, collections	Additional query/queries and Python-side matching

They can be combined:

```
Post.objects.select_related(
    "author"
).prefetch_related("tags")
```

This assumes `Post` has a `tags` relationship.

বাংলা: N+1 query কমাতে সম্পর্ক অনুযায়ী `select_related()` বা `prefetch_related()` ব্যবহার করো। Query count মেপে ফলাফল যাচাই করবে।

Chapter 12: Aggregation and Annotation

12.1 `aggregate()`

`aggregate()` returns summary values for the whole QuerySet.

```
from django.db.models import Avg, Max, Min, Sum

result = Student.objects.aggregate(
    average_age=Avg("age"),
    oldest=Max("age"),
    youngest=Min("age"),
)
```

It returns a dictionary. The values depend on the actual data.

12.2 `annotate()`

`annotate()` adds calculated values to each returned object.

```
from django.db.models import Count

authors = Author.objects.annotate(
    post_count=Count("posts")
)

for author in authors:
    print(author.name, author.post_count)
```

12.3 Filter by Annotation

```
authors = (
    Author.objects
    .annotate(post_count=Count("posts"))
```

```
    .filter(post_count__gte=5)
)
```

Some joins can duplicate rows. Depending on the query, `Count("posts", distinct=True)` may be needed.

বাংলা: `aggregate()` পুরো QuerySet-এর summary দেয়। `annotate()` প্রতিটি object-এর সঙ্গে calculated value যোগ করে।

Chapter 13: F Expressions

13.1 What is `F()`?

An `F()` expression refers to a database field inside a query. It allows database-side calculations and field comparisons.

```
from django.db.models import F
```

Example 1: Decrease stock

```
Product.objects.filter(
    stock__gt=0
).update(
    stock=F("stock") - 1
)
```

Conceptually, the database executes an update like:

```
UPDATE products_product
SET stock = stock - 1
WHERE stock > 0;
```

The calculation happens in the database instead of first reading the old value into Python. This can avoid common read-modify-write race conditions. More complex inventory rules may still need transactions, constraints, or locking.

Example 2: Compare fields

```
Employee.objects.filter(
    salary__gt=F("manager__salary")
)
```

Assuming `Employee` has a `manager` relationship, this compares an employee's salary with their manager's salary.

Example 3: Use F on an instance

```
product = Product.objects.get(pk=1)

product.stock = F("stock") - 1
product.save(update_fields=["stock"])
product.refresh_from_db()
```

Refresh the instance before treating the field as a normal Python value.

Interview answer: `F()` refers to a database field in a query and enables database-side calculations and field comparisons.

বাংলা: `F()` ব্যবহার করলে পুরোনো value Python-এ এনে calculation করার বদলে database-কে সরাসরি calculation করতে বলা যায়।

Chapter 14: Conditional Expressions

14.1 `Case()` and `When()`

```
from django.db.models import Case, When, Value, CharField

students = Student.objects.annotate(
    age_group=Case(
        When(age__lt=18, then=Value("minor")),
        default=Value("adult"),
        output_field=CharField(),
    )
)
```

Each returned Student gets an `age_group` calculated according to the condition.

- `When()` : condition.
- `then` : value when the condition matches.
- `default` : fallback value.
- `output_field` : output data type.

14.2 Database Functions

```
from django.db.models.functions import Lower, Upper, Length

students = Student.objects.annotate(
    lower_name=Lower("name"),
    upper_name=Upper("name"),
    name_length=Length("name"),
)
```

These functions calculate lowercase text, uppercase text, and string length in the database.

14.3 `Coalesce()`

`Coalesce()` returns the first non-NULL expression.

```
from django.db.models import Value, CharField
from django.db.models.functions import Coalesce

students = Student.objects.annotate(
    display_name=Coalesce(
        "nickname",
        "name",
        Value("Unknown", output_field=CharField()),
    )
)
```

This assumes `Student` has a nullable `nickname` field. The expressions should have compatible data types.

বাংলা: Conditional expressions দিয়ে SQL query-এর মধ্যেই শর্ত অনুযায়ী value তৈরি করা যায়।

Chapter 15: Subquery and Exists

15.1 `Exists()`

Use `Exists()` to check whether related records exist.

```
from django.db.models import Exists, OuterRef

posts = Post.objects.filter(
    author=OuterRef("pk")
)

authors = Author.objects.annotate(
    has_posts=Exists(posts)
)
```

`OuterRef("pk")` refers to the primary key of the outer Author query. `has_posts` indicates whether a matching Post exists.

To return only authors with posts:

```
authors = Author.objects.filter(Exists(posts))
```

15.2 `Subquery()`

Get the latest Post title for each Author:

```
from django.db.models import OuterRef, Subquery

latest_post = (
    Post.objects
    .filter(author=OuterRef("pk"))
    .order_by("-created_at", "-pk")
)

authors = Author.objects.annotate(
    latest_post_title=Subquery(
        latest_post.values("title")[:1]
    )
)
```

This assumes `Post` has a `created_at` field.

Explanation:

1. `OuterRef("pk")` identifies the current Author.
2. `filter()` selects that author's Posts.
3. `order_by()` puts the latest Post first.
4. `values("title")` selects one column.
5. `[:1]` limits the subquery to one row.

The subquery must return the number of rows and columns required by the expression.

বাংলা: `Exists()` সম্পর্কিত record আছে কি না পরীক্ষা করে। `Subquery()` অন্য query-এর result-কে আরেকটি query-তে ব্যবহার করতে দেয়।

Chapter 16: Managers and Custom QuerySets

16.1 What is a Manager?

A Manager provides an interface for database queries on a model. `objects` is the common default Manager.

```
Student.objects.all()
```

16.2 Custom QuerySet

```
from django.db import models
```

```
class PublishedPostQuerySet(models.QuerySet):
    def published(self):
        return self.filter(is_published=True)

    def by_author(self, author):
        return self.filter(author=author)


class Post(models.Model):
    title = models.CharField(max_length=200)
    is_published = models.BooleanField(default=False)

    objects = PublishedPostQuerySet.as_manager()
```

Usage:

```
Post.objects.published()
Post.objects.published().by_author(author)
```

Be careful when adding default filters to a Manager. They can hide records from admin pages or other workflows.

বাংলা: Reusable query logic custom QuerySet-এ রাখলে code পরিষ্কার হয় এবং একই query বারবার লেখার প্রয়োজন কমে।

Chapter 17: Database Transactions

17.1 `transaction.atomic()`

A transaction groups database operations into one logical unit.

```
from django.db import transaction

with transaction.atomic():
    order = Order.objects.create(
        customer=customer,
        total=1000,
    )

    OrderItem.objects.create(
        order=order,
        product=product,
        quantity=2,
    )
```

This assumes the `Order` and `OrderItem` models have the required fields.
If an uncaught exception causes the transaction to roll back, the database changes in that transaction can be undone. External actions such as sending an email cannot automatically be rolled back.

17.2 Why Use Transactions?

Suppose an order is created but inventory is not updated. A transaction helps avoid unwanted partial database changes. Concurrency control and constraints may also be necessary.

17.3 `transaction.on_commit()`

```
from django.db import transaction

transaction.on_commit(
    lambda: print("Transaction committed")
)
```

The callback runs after a successful commit. For important background work, a durable task queue or transactional outbox pattern may be needed because a process can stop after commit but before the callback's work completes.

বাংলা: একাধিক সম্পর্কিত database operation একসঙ্গে সফল বা ব্যর্থ হওয়া দরকার হলে transaction ব্যবহার করো।

Chapter 18: Raw SQL

Use the ORM for most queries. Raw SQL can help with special or database-specific queries.

18.1 `raw()`

```
students = Student.objects.raw(
    """
    SELECT *
    FROM students_student
    WHERE age >= %s
    """,
    [18],
)
```

The table name must match the actual database schema. A raw model query normally needs the model's primary key in the selected columns.

18.2 Cursor

```
from django.db import connection

with connection.cursor() as cursor:
    cursor.execute(
        """
        SELECT COUNT(*)
        FROM students_student
        WHERE age >= %s
        """,
        [18],
    )

    row = cursor.fetchone()
    print(row[0])
```

Use parameterized queries. Never join untrusted user input directly into an SQL string.

```
# Unsafe pattern — do not use
# query = f"SELECT * FROM students WHERE name = '{user_input}'"
```

বাংলা: Raw SQL ব্যবহার করলেও parameterized query ব্যবহার করতে হবে, নইলে SQL injection-এর ঝুঁকি তৈরি হতে পারে। Dynamic field names-এর জন্য allowlist দরকার।

Chapter 19: Query Optimization and Indexes

19.1 Query Plan

```
students = Student.objects.filter(age__gte=18)
print(students.explain())
```

`explain()` helps inspect how the database plans to execute a query. It may show scans, index use, joins, and other operations. The plan depends on the database, data, and configuration.

19.2 When to Use Indexes

Consider indexes for:

- Frequently filtered fields.
- Join keys.
- Frequently ordered fields.
- Common combinations of fields used by queries.

Do not add indexes to every field. Indexes use storage and can slow down writes.

19.3 Composite Index

```
class Student(models.Model):
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        indexes = [
            models.Index(
                fields=["is_active", "created_at"],
                name="student_active_created_idx",
            ),
        ]
```

The order of fields in a composite index matters. A B-tree composite index is often most useful for queries matching its leading fields, but real query plans should guide decisions.

19.4 Optimization Checklist

1. Reduce N+1 queries.
2. Use `select_related()` and `prefetch_related()` where appropriate.
3. Load only the fields you need.
4. Paginate large result sets.
5. Measure query counts and response times.
6. Inspect query plans.
7. Add indexes based on real query patterns.

বাংলা: অনুমান করে optimization না করে query count, response time ও execution plan মেপে সিদ্ধান্ত নাও।

Chapter 20: Bulk Operations

Bulk operations process many records efficiently.

20.1 `bulk_create()`

```
students = [
    Student(name="A", email="a@example.com"),
    Student(name="B", email="b@example.com"),
]

Student.objects.bulk_create(
    students,
    batch_size=500,
)
```

20.2 `bulk_update()`

```
students = list(
    Student.objects.filter(is_active=False)
)

for student in students:
    student.is_active = True

Student.objects.bulk_update(
    students,
    ["is_active"],
    batch_size=500,
)
```

Bulk operations generally do not call each instance's `save()` method or normal per-instance save signals. Backend behavior can vary.

Feature	**`save()`**	**Bulk operations**
Instance save behavior	Runs	Usually bypassed
Normal save signals	Applicable	Usually not sent per instance
Many records	Can have more overhead	Often more efficient
Per-object custom logic	Straightforward	May need separate handling

বাংলা: অনেক record একসঙ্গে insert/update করতে bulk method কার্যকর, তবে validation ও business logic কোথায় হবে তা নিশ্চিত করতে হবে।

Chapter 21: Multiple Databases

Django can connect to multiple databases.

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    },
    "analytics": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "analytics.sqlite3",
    },
}
```

Read from a specific database:

```
Student.objects.using("analytics").all()
```

Save to a specific database:

```
student.save(using="analytics")
```

Multiple databases may require database routers and careful migration planning. Cross-database relationships and transactions have limitations; one atomic transaction does not automatically cover every database.

বাংলা: একাধিক database থাকলে কোন model কোথায় read/write হবে, migration কীভাবে চলবে এবং transaction-এর সীমাবদ্ধতা কী—এসব পরিকল্পনা করতে হয়।

Chapter 22: Fixtures

Fixtures are files containing data that can be loaded into a database.

Export data:

```
python manage.py dumpdata students --indent 2 > students.json
```

Import data:

```
python manage.py loaddata students.json
```

Fixtures can help create sample data, demo records, or known test data.

Consider primary keys, model dependencies, and environment-specific data. Remove private information before sharing exports.

বাংলা: Fixture development ও testing-এ নির্দিষ্ট data পুনরায় load করার কাজে লাগে। Sensitive production data অসতর্কভাবে export/share করবে না।

Chapter 23: Signals

Signals let a function respond to certain events. For example, `post_save` is sent after a model instance is saved.

```
from django.db.models.signals import post_save
from django.dispatch import receiver
from .models import Student


@receiver(post_save, sender=Student)
def student_saved(sender, instance, created, **kwargs):
    if created:
        print(f"New student created: {instance.pk}")
```

- `post_save` : signal sent after saving.
- `sender` : model that sent the signal.
- `instance` : saved object.
- `created` : whether a new object was created.

Register signal handlers through an appropriate app configuration, commonly by importing the signal module from `AppConfig.ready()` .

Best practices:

- Avoid hiding complex business logic in signals.
- Keep receivers easy to test and safe to run more than once where possible.
- Avoid duplicate registration.
- `QuerySet.update()` does not call normal save signals.
- Bulk operations generally do not call per-instance save signals.
- Explicit service functions may be easier to understand and test.

বাংলা: Signal event-এর পরে কাজ চালাতে পারে, কিন্তু সব business logic signal-এর মধ্যে রাখলে debugging কঠিন হতে পারে।

Chapter 24: Validation, Constraints and Security

24.1 Model Validation

```
student = Student(
    name="",
    email="invalid-email",
)

student.full_clean()
```

`full_clean()` performs field, model, and constraint-related validation where applicable, and may raise `ValidationError`.

Django's `save()` does not automatically call `full_clean()`. Decide where validation should occur. In Django REST Framework applications, serializer validation is also important.

24.2 `CheckConstraint`

Example: Product price must not be negative.

```
from django.db import models


class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )

    class Meta:
        constraints = [
            models.CheckConstraint(
                condition=models.Q(price__gte=0),
                name="product_price_nonnegative",
            )
        ]
```

This creates a database-level rule that the price must be at least zero.

24.3 `UniqueConstraint`

Prevent duplicate enrollment:

```
class Enrollment(models.Model):
    learner = models.ForeignKey(
        "Learner",
        on_delete=models.CASCADE,
    )
```

```
    course = models.ForeignKey(
        "Course",
        on_delete=models.CASCADE,
    )

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["learner", "course"],
                name="unique_enrollment",
            )
        ]
```

A database constraint protects data integrity even when concurrent requests try to create duplicates.

24.4 Security Checklist

- Use parameterized queries for raw SQL.
- Check user permissions and object-level authorization.
- Keep credentials out of source code.
- Set `DEBUG=False` in production.
- Configure `ALLOWED_HOSTS` correctly.
- Do not log sensitive information.
- Give database users only the permissions they need.
- Test backup and restore procedures.
- Allowlist user-selectable ordering fields and column names.
- Test authentication and authorization on protected API endpoints.

বাংলা: শুধু form বা serializer validation যথেষ্ট নয়। Data integrity-এর জন্য প্রয়োজন অনুযায়ী database constraint-ও ব্যবহার করো।

Chapter 25: Django ORM Interview Questions

Beginner Level

1. What is Django ORM?

It lets you work with a relational database through Python models and query methods.

2. What is a Model?

A Python class that defines the structure and behavior of stored data.

3. What is a Migration?

A version-controlled instruction for changing database schema.

4. Difference between `makemigrations` and `migrate`?

`makemigrations` creates migration files. `migrate` applies them to the database.

5. Difference between `get()` and `filter()`?

`get()` expects one object and may raise an exception. `filter()` returns a QuerySet.

6. What is a lazy QuerySet?

A QuerySet usually does not run its database query until its results are needed.

Intermediate Level

7. Difference between `select_related()` and `prefetch_related()`?

`select_related()` uses SQL JOINs for single-valued relationships.

`prefetch_related()` generally uses additional queries for related collections.

8. Difference between `annotate()` and `aggregate()`?

`annotate()` adds calculated values to individual objects. `aggregate()` returns a summary for the whole QuerySet.

9. What is an F expression?

It refers to a database field inside a query and allows database-side calculations or field comparisons.

10. What is a Q object?

It builds complex query conditions such as OR, AND, and NOT.

11. Difference between `null=True` and `blank=True`?

`null` affects database storage. `blank` affects validation.

12. What does CASCADE do?

It causes related objects to be deleted when the referenced object is deleted, according to the relationship's deletion behavior.

13. Should every field have an index?

No. Indexes use storage and may slow down writes. Add them based on query patterns and measurements.

Advanced Level

14. What is `transaction.atomic()`?

It groups database operations in a transaction so appropriate changes can roll back on failure.

15. Difference between `update()` and `save()`?

`update()` directly updates matching rows. `save()` saves a model instance.

16. What is the N+1 query problem?

It occurs when related data triggers extra queries for many individual objects.

17. Why use `bulk_create()`?

It can insert many records efficiently, but does not provide all normal per-instance save behavior.

18. Why use database constraints?

They protect data integrity at the database level, including against many concurrent-request problems.

19. When should you use `exists()`?

When you only need to know whether any matching record exists.

20. How do you optimize ORM queries?

Measure query counts and response times, inspect query plans, reduce N+1 queries, load only needed data, paginate results, and add indexes carefully.

Chapter 26: Quick ORM Cheat Sheet

```
# CREATE
Student.objects.create(
    name="Rahim",
    email="rahim@example.com",
)

# READ
Student.objects.all()
Student.objects.get(pk=1)
Student.objects.filter(age__gte=18)

# UPDATE
Student.objects.filter(pk=1).update(
    is_active=False
)

# DELETE
Student.objects.filter(pk=1).delete()

# ORDERING
```

```
Student.objects.order_by("-created_at")

# LIMIT
Student.objects.all()[:10]

# EXISTS
Student.objects.filter(
    is_active=True
).exists()

# COUNT
Student.objects.count()

# OR CONDITION
from django.db.models import Q

Student.objects.filter(
    Q(age__lt=18) | Q(age__gt=60)
)

# F EXPRESSION
from django.db.models import F

Product.objects.filter(
    stock__gt=0
).update(stock=F("stock") - 1)

# AGGREGATION
from django.db.models import Count

Author.objects.aggregate(total=Count("id"))

# ANNOTATION
Author.objects.annotate(post_count=Count("posts"))

# SELECT RELATED
Post.objects.select_related("author")

# PREFETCH RELATED
Author.objects.prefetch_related("posts")

# TRANSACTION
from django.db import transaction
```

```
with transaction.atomic():
    pass
```

Migration commands:

```
python manage.py makemigrations
python manage.py migrate
python manage.py showmigrations
```

বাংলা: এই cheat sheet coding-এর সময় দ্রুত reference হিসেবে ব্যবহার করতে পারো। তবে প্রতিটি method-এর behavior ও সীমাবদ্ধতা বুঝে ব্যবহার করবে।

Chapter 27: Learning Roadmap

Beginner Level

1. Learn tables, rows, columns, and primary keys.
2. Learn foreign keys.
3. Create Django models and fields.
4. Practice migrations.
5. Practice CRUD operations.
6. Write queries in the Django shell.

Intermediate Level

1. Learn field lookups and Q objects.
2. Practice ForeignKey, OneToOne, and ManyToMany relationships.
3. Learn F expressions.
4. Practice aggregation and annotation.
5. Use `select_related()` and `prefetch_related()`.
6. Learn validation and database constraints.
7. Practice transactions.

Advanced Level

1. Understand query plans and indexes.
2. Learn Subquery and Exists.
3. Practice bulk operations.
4. Create custom managers and QuerySets.
5. Learn multiple databases and routers.
6. Understand concurrency and transaction isolation.

7. Plan production migrations.
8. Monitor query performance.

Capstone Project: Student Course Management API

Build a practical backend project that combines these concepts.

Models

- Student
- Course
- Instructor
- Enrollment

Requirements

- Student CRUD API
- Course CRUD API
- Student enrollment system
- Prevent duplicate enrollments
- Count students in each course
- List courses for each student
- Filtering and pagination
- Query optimization
- Transaction support
- Automated tests
- PostgreSQL configuration
- API documentation

Example: Count Students per Course

Assuming `Learner.courses` is a ManyToManyField with `related_name="learners"`:

```
from django.db.models import Count

courses = Course.objects.annotate(
    student_count=Count("learners", distinct=True)
)
```

Example: Load a Learner's Courses Efficiently

```
learner = Learner.objects.prefetch_related(
    "courses"
).get(pk=1)
```

```
for course in learner.courses.all():
    print(course.title)
```

Test Cases

- Duplicate email is rejected.
- Duplicate enrollment is prevented.
- Missing records are handled correctly.
- Unauthorized users cannot access protected data.
- N+1 queries are reduced.
- Failed transactions do not leave unwanted partial database changes.

Official Django Documentation

- Django 6.0 Documentation: https://docs.djangoproject.com/en/6.0/
- Models: https://docs.djangoproject.com/en/6.0/topics/db/models/
- QuerySets: https://docs.djangoproject.com/en/6.0/ref/models/querysets/
- Query Expressions: https://docs.djangoproject.com/en/6.0/ref/models/expressions/
- Database Optimization: https://docs.djangoproject.com/en/6.0/topics/db/optimization/
- Transactions: https://docs.djangoproject.com/en/6.0/topics/db/transactions/
- Migrations: https://docs.djangoproject.com/en/6.0/topics/migrations/

Final Advice

Reading notes is not enough to master Django ORM.

1. **Practice:** Write every example yourself.
2. **Debug:** Inspect generated SQL and query counts.
3. **Test:** Check edge cases, constraints, concurrent updates, and transaction failures.
4. **Optimize:** Measure query performance using realistic data.
5. **Build:** Apply the concepts in a complete project.

Practice + Testing + Debugging = Strong Django ORM Skills

বাংলা: শুধু code মুখস্থ করবে না। কেন ব্যবহার করতে হয়, কখন ব্যবহার করতে হয়, database-এ কী প্রভাব পড়ে এবং কীভাবে test করতে হয়—এসব বুঝলেই Django ORM-এ দক্ষতা তৈরি হবে।